LOTL-ARIA: An Agentic LLM Framework for Autonomous Linux Living-off-the-Land Attack Investigation and Response

Introduction

1.1 Introduction

This chapter establishes the theoretical foundation upon which the proposed research is constructed. It begins by examining the role of Artificial Intelligence (AI) in modern cybersecurity and the emergence of autonomous agents for security operations. The chapter then defines Living-off-the-Land (LOTL) attacks, explains their significance, and presents the GTFOBins catalog as the primary source of Linux LOTL primitives. Subsequently, it analyzes the limitations of rule-based detection and machine learning classifiers for command-line analysis, with particular attention to the LOTLDetector system, which serves as the foundational paper for this project [1]. The chapter then introduces Large Language Models (LLMs) and the ReAct framework as a paradigm for autonomous investigation and response, discusses prompt injection threats against LLM-based security agents, and concludes by articulating the research gap that this project addresses [1].

1.2 Artificial Intelligence in Cybersecurity: General and Specific Significance

1.2.1 General Significance of the Specialization

The integration of Artificial Intelligence into cybersecurity represents one of the most consequential technological shifts of the current decade. Security Operations Centers (SOCs) face an operational crisis driven by the exponential growth in alert

volume, the global shortage of skilled cybersecurity personnel, and the increasing sophistication of adversarial techniques. A typical mid-size SOC generates thousands of security alerts every day, with false-positive rates approaching 99% in many environments. Studies have demonstrated that 30–50% of a Tier-1 analyst's time is consumed by repetitive triage tasks, leading to alert fatigue, burnout, and delayed response to genuine threats [2].

Artificial Intelligence—encompassing machine learning, deep learning, generative models, and autonomous agents—offers a pathway to augment human analytical capacity, automate repetitive investigative tasks, and improve the speed and consistency of incident response. The strategic importance of AI in cybersecurity is reflected in national cyber defense strategies, industry frameworks, and academic research agendas worldwide [3].

1.2.2 Specific Significance of the Specialization

Within the broader domain of AI-driven cybersecurity, this project focuses on a specific and acute problem: the detection and response to Living-off-the-Land (LOTL) attacks on Linux systems. LOTL attacks exploit legitimate system binaries—tools already present on the target system—to conduct malicious activities, thereby evading signature-based detection and blending into normal administrative operations [1]. The LOTLDetector paper, published in Cybersecurity (2026),

represents the current state-of-the-art in detecting these attacks using a hybrid of deep learning and expert rules, achieving an F1-score of 0.9727 on a Linux dataset [1]. However, the authors explicitly acknowledge two fundamental limitations: the absence of command-line context and the lack of model interpretability [1]. These limitations define the specific gap this project addresses.

1.3 Research Gap

The literature reviewed in this chapter reveals a clear and unarguable research gap. Rule-based detection systems are rigid, easily bypassed by obfuscation, and cannot handle the dual-use problem [1]. ML-based classifiers achieve high in-distribution accuracy but suffer from poor generalization, lack of explainability, and an inability to investigate context or execute response actions [1]. The state-of-the-art LOTLDetector system explicitly acknowledges these limitations and identifies LLM-based agents with process tree analysis and natural language explanation generation as its intended future direction [1].

Specifically, LOTLDetector, the state-of-the-art detection system, achieves high classification accuracy in-distribution but suffers from three critical deficiencies [1]:

1. **Lack of Contextual Understanding:** The model evaluates commands in isolation, without access to process trees, user

behavior histories, session activity, or asset criticality. This leads to misclassification of legitimate administrative commands as malicious (false positives) and vice versa (false negatives) [1].

2. **Absence of Explainability:** SHAP and LIME, the standard explainability techniques, fail on high-dimensional Word2Vec vectors. The model cannot explain why a command is classified as malicious, undermining analyst trust and hindering post-incident review [1].

3. **No Response Capability:** The system is detection-only. It cannot investigate, propose containment actions, or escalate incidents. Human analysts must manually perform all post-detection work [1].

The authors of LOTLDetector explicitly state that their future work will be centered around Large Language Models (LLMs) to address these limitations, by transforming process trees and user behavior logs into semantic text that LLMs can parse, utilizing LLM reasoning capabilities to trace process source chains, and automatically generating natural language explanations [1].

However, no existing work has implemented and empirically validated an LLM agent that investigates and responds to Linux LOTL attacks. No work has compared such an agent's

generalization to unseen binaries and obfuscation against ML classifiers and published rules. No work has tested the robustness of a LOTL detection agent to prompt injection hidden in the command line [4][13][14]. This project addresses all three of these gaps.

The proposed system builds upon the ReAct framework to create an autonomous agent that: (1) ingests raw command events from Linux auditd and sudo logs via Wazuh; (2) investigates context using a set of specialized tools including user baseline analysis, process tree traversal, and GTFOBins catalog lookup; (3) generates dual hypotheses (malicious vs. benign) and seeks evidence to distinguish between them; (4) proposes graded response actions ranging from monitoring to escalation; and (5) validates every action through a safety gate that blocks harmful operations and requires human approval for high-impact actions [20]. The system is evaluated against published rules, an ML classifier, and a zero-shot LLM baseline on datasets including the QuasarNix reverse shell corpus, GTFOBins sudo escapes, and benign twins [1][19].

1.4 Project Goal

1.4.1 Summary of the Foundational Paper

The foundational paper for this project is "Lotldetector: living off the land attacks detection system based on feature fusion" by Zhu et al., published in Cybersecurity (2026) [1]. This paper

addresses the problem of detecting malicious command lines used in LOTL attacks on both Windows and Linux platforms. The authors propose LOTLDetector, a detection system that integrates deep learning with expert rules to achieve high-precision classification of command lines into three categories: normal, suspicious, and malicious.

The system's architecture comprises four stages: data preprocessing (including multi-layer de-obfuscation), feature extraction (token extraction, rule extraction, and similarity comparison), feature encoding (Word2Vec embeddings for tokens, rules, and similarity labels), and feature learning (a combined Transformer-TextCNN model). The system was evaluated on a Windows dataset containing 27,448 command lines and a Linux dataset containing 27,093 command lines, achieving an F1-score of 0.9727 on the Linux dataset and 0.9598 on the Windows dataset [1].

Crucially, the authors acknowledge two limitations that define the research gap for this project. First, they state that "the current lack of command-line context leads to a limitation in our system," noting that many commands used maliciously are also used by ordinary users for legitimate purposes [1]. Second, they note that "the interpretability of model results is undoubtedly one of the important directions for our future research work," observing that SHAP and LIME fail on high-dimensional Word2Vec vectors [1].

The authors' stated future work directly addresses these limitations: "In the future, our subsequent plans will be centered around Large Language Models (LLM) implementation, by transforming the hierarchical relationships of process trees or user behavior logs into semantic text that LLMs can parse, utilizing LLM's reasoning capabilities to trace the source chain of processes, and automatically generating natural language explanations" [1].

1.4.2 Summary of What This Project Seeks to Implement

This project builds directly upon the future work identified by Zhu et al. [1]. The objective is to design, implement, and empirically validate an Agentic LLM Approach to Linux Living-off-the-Land Attack Detection and Response, benchmarked against machine learning classifiers and published rules. The system will:

1. Ingest raw command events from Linux auditd and sudo logs via Wazuh SIEM.

2. Investigate context using specialized tools, including user baseline analysis, process tree traversal, session activity review, and GTFOBins catalog lookup.

3. Generate dual hypotheses (malicious vs. benign) and actively seek evidence to distinguish between them.

4. Propose graded response actions ranging from monitoring to session termination, privilege revocation, account lockout, host isolation, and escalation to a human Tier-2 analyst.

5. Validate every action through a safety gate that blocks harmful operations, requires human approval for high-impact actions, and executes reversible actions autonomously.

6. Generate incident reports with complete reasoning traces and natural language explanations.

7. The system will be evaluated against three baselines: published rules (Splunk Linux LOTL detections), an ML classifier (LOTLDetector or a comparable model), and a zero-shot LLM. The evaluation will measure detection performance (Precision, Recall, F1), response appropriateness (correct-action rate, harmful-action rate), prompt injection robustness (injection success rate with and without the safety gate), and operational metrics (latency, cost per command).

1.5 Literature Review

This section reviews eleven research papers published in 2025 or 2026, excluding the foundational LOTLDetector paper [1]. Each review summarizes the work accomplished and its relationship to the present project.

1.5.1 RACONTEUR: LLM-Based Shell Command Explanation (NDSS 2025)

RACONTEUR is an LLM-powered tool that generates knowledgeable and insightful explanations for shell commands, including their intent and associated MITRE ATT&CK techniques [3]. The system addresses the problem that malicious commands often resemble benign ones, making it difficult for analysts to determine intent from the command line alone. RACONTEUR was evaluated on a dataset of real-world commands and demonstrated the ability to identify malicious intent that signature-based tools miss.

Relationship to this project: RACONTEUR provides command explanation but stops short of investigation and response. It is a passive advisory tool. This project extends the concept by integrating explanation into an active investigation and response agent.

1.5.2 Automated Risk Assessment of Shell-Based Attacks Using LLM (Springer 2025)

This paper proposes an LLM-based classifier that assesses the risk of shell commands into five distinct levels (R0 to R4), moving beyond binary classification [5]. The system evaluates commands based on their potential impact and generates a structured risk score that can inform response prioritization. The authors demonstrate that LLM-based risk assessment correlates with expert human judgment.

Relationship to this project: This work provides the foundation for graded response actions. This project adopts a similar multi-level risk assessment approach but extends it by connecting risk scores to concrete response actions (monitor, kill session, revoke sudo, lock account, isolate host).

1.5.3 Leveraging LLMs for Non-Security Experts in Threat Hunting: Detecting LotL (MDPI 2025)

This paper evaluates whether LLMs such as ChatGPT can generate effective queries for threat hunting of LOTL techniques [6]. The authors conclude that "LLMs do not consistently produce accurate or reliable queries for detecting these techniques" and that "they should not be relied upon for accurate results in threat detection and require further refinement to be effectively integrated into cybersecurity workflows."

Relationship to this project: This paper is the closest prior work on LLMs and LOTL and provides direct justification for this project. Its conclusion—that simple LLM query generation is unreliable—supports the need for a more sophisticated agentic system with investigation tools and hypothesis testing.

1.5.4 SENTINEL: LOTL Failure Modes (MILCOM 2026)

SENTINEL analyzes why current LOTL detection systems fail, focusing on Windows command lines [7]. The paper identifies three critical failure modes: obfuscation, intent ambiguity, and multi-stage attacks. The authors propose a multi-pathway architecture that addresses these failure modes through parallel detection strategies.

Relationship to this project: SENTINEL's identification of failure modes validates the need for the held-out split and obfuscated variants in this project's evaluation. It also supports the hypothesis that LLM agents, with their ability to reason about intent, may outperform rule-based and ML systems on obfuscated and multi-stage attacks.

1.5.5 Supervised Learning for LOLBins Detection in Linux Machines (Springer 2026)

This paper compares multiple supervised learning techniques for detecting LOLBins attacks on Linux systems [8]. The authors find that Decision Tree and Random Forest classifiers outperform other methods but highlight a critical limitation: class imbalance and the difficulty of obtaining accurately labeled malicious data. The paper concludes that future work must explore unsupervised or semi-supervised learning approaches that do not rely on perfectly labeled malicious data.

Relationship to this project: This paper's findings support the use of LLM agents, which can reason about commands without requiring large labeled datasets. The class imbalance problem identified here is addressed by the agent's ability to investigate context and distinguish between benign and malicious intent.

1.5.6 BlueSTAR: Rules-First Tiered Architecture (2026)

BlueSTAR proposes a tiered architecture for security automation that combines rule-based detection with LLM-based reasoning [9]. The system first applies fast, deterministic rules to filter known patterns, then routes uncertain cases to an LLM for deeper analysis. This hybrid approach balances speed and accuracy, achieving low latency for common cases while providing reasoning for complex ones.

Relationship to this project: BlueSTAR's rules-first design directly inspires the architecture of this project. The proposed system adopts a similar tiered approach: known patterns are handled by published rules, while uncertain cases are routed to the LLM agent for investigation.

1.5.7 AgentSOC: Multi-Layer Agentic AI Framework for Security Operations (2026)

AgentSOC introduces a multi-layered agentic AI framework for SOC automation with a Narrative Counterfactual Engine and Structural Simulation for risk-based action planning [10]. The system generates multiple plausible attack progressions using LLM reasoning grounded in MITRE ATT&CK, then validates whether recommended response actions are operationally feasible. However, AgentSOC's evaluation is limited to dry-run mode using LANL authentication data—it does not connect to live execution APIs.

Relationship to this project: AgentSOC represents the closest architectural parallel to this project. However, its dry-run limitation is precisely the gap this project addresses by implementing live execution with a safety gate. The counterfactual reasoning approach of AgentSOC informs the hypothesis generation mechanism in this project.

1.5.8 Hyper-Automation SOAR with Agentic AI (MDPI 2025)

This paper explores the integration of agentic AI with Security Orchestration, Automation, and Response (SOAR) platforms [11]. The authors propose a hyper-automation framework in which LLM agents generate, validate, and execute response playbooks dynamically, rather than relying on static, pre-written playbooks. The system demonstrates improved adaptability to novel threats but acknowledges challenges in safety and governance.

Relationship to this project: This work supports the shift from static SOAR playbooks to dynamic, agent-driven response. The safety challenges identified by the authors are addressed in this project through the safety gate mechanism.

1.5.9 GIRA: Safety Gate for Incident Response Agents (ICLR 2026 Workshop)

GIRA proposes a safety gate architecture for incident response agents that validates every proposed action against a set of hard-coded safety rules before execution [12]. The safety gate categorizes actions into three tiers: reversible actions (executed autonomously), high-impact actions (require human approval), and dangerous actions (categorically prohibited). The authors demonstrate that this architecture significantly reduces harmful actions without impairing system utility.

Relationship to this project: GIRA directly informs the safety gate mechanism in this project. The proposed system adopts a similar three-tier categorization of actions, with additional safeguards specifically designed to address prompt injection hidden in command lines.

1.5.10 Poisoning the Watchtower: Log-Based Prompt Injection Against SOC LLMs (2026)

This paper demonstrates that SOC LLMs are vulnerable to prompt injection attacks embedded in log data [13]. The authors show that an attacker can craft log entries that manipulate the LLM's reasoning, causing it to misclassify malicious activity as benign or to execute harmful response actions. The paper evaluates multiple defense strategies and finds that none are fully effective, concluding that defense-in-depth is required.

Relationship to this project: This paper provides the threat model for the prompt injection experiments in this project. It demonstrates that prompt injection is a real and unsolved threat to LLM-based SOC systems, justifying the need for a safety gate. The proposed safety gate is evaluated specifically against the attack vectors identified in this paper.

1.5.11 LogJack: Injected Commands Executed by Tool-Using Agents (2026)

LogJack extends the prompt injection threat to tool-using LLM agents, demonstrating that injected commands in log data can cause agents to invoke dangerous tools [14]. The paper shows that even agents with restricted tool sets can be manipulated into executing unintended actions. The authors propose tool-level access controls as a partial mitigation.

Relationship to this project: LogJack's findings reinforce the need for the safety gate in this project. The proposed safety gate operates at the tool-execution level, validating every action against hard-coded safety rules before execution, regardless of the agent's reasoning.

1.5.12 Summary of Previous Studies

#	Paper Title	Author(s)	Publisher	Year	Relationship to Project
1	RACONTEUR: LLM-Based Shell Command Explanation	Jiangyi Deng, Xinfeng Li, Yanjiao Chen, Yijie Bai, Haiqin Weng, Yan Liu, Tao Wei, Wenyuan Xu	NDSS Symposium	2025	Provides command explanation; passive advisory only [4]
2	Automated Risk Assessment of		Springer		Foundation for graded response actions [5]

	Shell-Based Attacks Using LLM	Sereysethy Touch, Jérôme Fink, Jean-Noël Colin		2025	
3	Leveraging LLMs for Non-Security Experts in Threat Hunting: Detecting LotL	A. Konstantinou, D. Kasimatis, W.J. Buchanan, S.U. Jan, J. Ahmad, I. Politis, N. Pitropakis	MDPI	2025	Closest LLM + LOTL work; query generation only [6]
4	SENTINEL: LOTL Failure Modes	Ahad Bin Islam Shoeb, Kamrul Hasan, Jamal Uddin Tanvin, Liang Hong, Imtiaz Ahmed, Md Arif Billah, Al Amin	IEEE	2026	Identifies failure modes; validates evaluation design [7]
5	Supervised Learning for LOLBins Detection in Linux Machines	Angello Cássio Vasconcelos Oliveira, Daniel Chaves Café	Springer	2024	Class imbalance warning; supports LLM approach [8]
6	BlueSTAR: Rules-First Tiered Architecture	Simona Boboila, Xavier Cadet, Edward	arXiv	2026	Inspires rules-first tiered architecture [9]

		Koh, Daniel Balasubramanian, Dirk Van Bruggen, Peter Chin, Alina Oprea			
7	AgentSOC: Multi-Layer Agentic AI Framework	Joyjit Roy, Samaresh Kumar Singh	IEEE	2026	Closest architectural parallel; dry-run limitation [10]
8	Hyper-Automation SOAR with Agentic AI	Ismail, Rahmat Kurnia, Zilmas Arjuna Brata, Ghitha Afina Nelistiani, Shinwook Heo, Hyeongon Kim	MDPI	2025	Supports dynamic SOAR; safety challenges [11]
9	GIRA: Safety Gate for Incident Response Agents	Dhruv Patel	ICLR Workshop	2026	Informs safety gate mechanism [12]
10	Poisoning the Watchtower: Log-Based Prompt Injection	Rohan Pandey, Archit Bhujang	arXiv	2026	Threat model for prompt injection experiments [13]
11	LogJack: Injected Commands Executed by Tool-Using Agents	Harsh Shah	arXiv	2026	Reinforces need for safety gate at tool level [14]

Table 1.1: Summary of Previous Studies (2025–2026)

1.6 Practical Applications

The practical applications of this project are directly relevant to Security Operations Centers, incident response teams, and organizations deploying Linux infrastructure. Specifically:

1. **Automated Tier-1/Tier-2 Investigation:** The agent autonomously investigates LOTL alerts, gathering context that would otherwise require manual analyst effort. This reduces Mean Time to Investigate (MTTI) and frees human analysts for complex, high-severity incidents.

2. **Context-Aware False Positive Reduction:** By understanding process trees, user behavior histories, and session activity, the agent distinguishes between legitimate administrative commands and malicious LOTL abuse. This addresses the dual-use problem that rule-based and ML systems cannot solve.

3. **Interpretable Decision-Making**: Every decision is accompanied by a natural language explanation, reasoning trace, and evidence summary. This satisfies governance, compliance, and audit requirements that black-box ML models cannot meet.

4. **Graded Autonomous Response:** The agent executes reversible actions (monitoring, session logging)

autonomously, proposes high-impact actions (account lockout, host isolation) for human approval, and blocks dangerous actions (file deletion, kernel modification) categorically.

5. **Prompt Injection Impact Mitigation:** The safety gate provides a deterministic validation layer that prevents adversarial manipulation of the agent's reasoning from translating into harmful actions, ensuring that even if the LLM is compromised, dangerous operations cannot execute.

6. **Open-Source, Reproducible Deployment:** The system is built on open-source components (Wazuh, GPT-OSS, Ollama, SQLite, LangGraph), making it accessible to academic researchers, small-to-medium organizations, and air-gapped environments.

1.7 Scientific Challenges

This project addresses several scientific challenges that are inherent to the integration of LLM agents with cybersecurity operations:

1. **Context Representation:** Transforming process trees, user behavior logs, and session activity into semantic text that LLMs can effectively parse and reason about. This requires

careful prompt engineering, structured data representation, and token budget management [1].

2. **Hypothesis Generation and Testing:** Designing prompts and tools that enable the agent to generate both malicious and benign hypotheses for a given command, and to actively seek evidence that distinguishes between them. This requires a structured investigation methodology [10].

3. **Generalization to Unseen Attacks:** Ensuring that the agent performs well on held-out binaries and obfuscated variants that were not seen during training or rule-writing. This is a fundamental challenge for LLM-based systems, which may have memorized public datasets [1][7].

4. **Prompt Injection Impact Mitigation:** Protecting the system (not the LLM itself) against adversarial manipulation embedded in command lines. This requires a deterministic safety gate and a defense-in-depth strategy [13][14].

5. **Safety and Governance:** Ensuring that the agent operates within well-defined boundaries, respects least-privilege access, and escalates appropriately to human analysts. This requires explicit safety rules, audit trails, and transparency mechanisms [12].

6. **Evaluation Methodology:** Designing a rigorous evaluation that compares the agent fairly against rule-based and ML baselines across multiple dimensions (detection, response, safety, cost). This requires careful dataset construction and metric selection [1][10].

7. **Latency and Cost:** Ensuring that the agent's reasoning and tool invocations complete within acceptable time and cost bounds. This requires efficient prompt design, local LLM inference, and parallel tool execution where possible [10][11].

Theoretical Foundations

2.1 Introduction

This chapter establishes the theoretical foundation for the proposed Agentic LLM Approach to Linux Living-off-the-Land (LOTL) Attack Detection and Response. It defines the core concepts of Living-off-the-Land attacks, examines the GTFOBins catalog as the primary attack surface for Linux privilege escalation, analyzes the limitations of existing rule-based and machine learning detection methods, and introduces Large Language Model (LLM) agents as a promising paradigm for autonomous security investigation and response. The chapter concludes by discussing the emerging threats to LLM-based security agents, including prompt injection, and the safety mechanisms required to deploy such systems in production environments.

2.2 Living-off-the-Land (LOTL) Attacks

Living-off-the-Land (LOTL) attacks represent a sophisticated class of cyber threats in which adversaries leverage legitimate, pre-installed system tools and binaries to conduct malicious activities [1]. Rather than deploying custom malware or exploiting zero-day vulnerabilities, attackers abuse trusted administrative utilities already present in the target environment, thereby evading signature-based detection and blending seamlessly into normal system operations.

LOTL techniques are widely employed in Advanced Persistent Threat (APT) campaigns. Research on Windows malware samples found that LOTL technology has a prevalence rate of 26.26% in APT malware samples [1]. A 2026 analysis of more than 700,000 cyber incidents found that 84% of major attacks leverage LOTL techniques, yet only 20.5% of surveyed organizations rank LOTL as a top-three threat—a 63.5 percentage-point gap between real-world prevalence and perceived importance [15]. These statistics underscore the urgent need for detection and response capabilities specifically designed to address LOTL threats.

LOTL attacks exhibit several defining characteristics that make them particularly challenging to detect. First, they produce a minimal attack footprint, as the tools used are already present on the system and require no additional binary deployment [1]. Second, they demonstrate persistence, as legitimate system utilities are unlikely to be removed or blocked by security teams. Third, they exploit dual-use tools—programs that have legitimate administrative purposes but can be repurposed for malicious ends [1].

2.3 GTFOBins: Catalog of Linux LOTL Primitives

GTFOBins is a curated, community-driven reference that documents how legitimate Unix binaries can be abused to break out of restricted environments, escalate privileges, read or write files, or spawn shells [16]. The project collects legitimate

functions of Unix binaries that can be abused to bypass local security restrictions in misconfigured systems [16]. GTFOBins is a collaborative project where everyone can contribute with additional binaries and techniques [16].

The GTFOBins catalog is particularly relevant because it explicitly documents the dual-use nature of system binaries. For example, find, a standard file-searching utility, can be abused with the -exec flag to spawn an interactive shell (sudo find . -exec /bin/sh \;). Similarly, vim, a text editor, can be used to escape restricted environments and execute arbitrary commands [16]. The catalog also includes less obvious binaries such as nano, awk, sed, and tar, each with documented abuse patterns.

The existence of GTFOBins highlights a fundamental challenge for defenders: these binaries cannot simply be blocked or removed, as they are essential for legitimate system administration. Detection must therefore focus on the context and intent of execution rather than the binary itself—precisely the capability that LLM agents are uniquely positioned to provide.

2.4 MITRE ATT&CK Framework for LOTL

The MITRE ATT&CK framework categorizes LOTL techniques under several tactics [17]. This project focuses on two primary techniques:

- **T1059.004 (Unix Shell):** Describes the abuse of Unix shells such as bash, sh, and zsh for command execution. This technique is commonly used in reverse shell attacks, where an attacker establishes a remote connection to a compromised system [17].

- **T1548.003 (Sudo and Sudo Caching):** Covers privilege escalation through misconfigured sudo permissions and shell escapes. This technique exploits the sudo command to execute commands with elevated privileges, often using GTFOBins primitives [17].

The MITRE ATT&CK framework provides a standardized vocabulary for describing attacker behavior, enabling consistent evaluation and comparison across detection systems [17].

2.5 Rule-Based Detection Methods

Traditional detection of malicious command-line activity relies on pattern-matching rules derived from expert knowledge [1]. These rule-based systems maintain a library of predefined signatures—specific command patterns, flags, or binary combinations—and flag any command that matches a known malicious pattern. Tools such as Splunk's Linux Living Off The Land detections and the Sigma rule format exemplify this approach [15].

Rule-based detection offers several advantages: it is transparent, computationally efficient, and produces no false positives for commands that match no rule. However, the limitations of this paradigm are well-documented and increasingly problematic. The primary weakness is that rule-based systems are fundamentally reactive—they can only detect patterns that have already been observed and documented. Novel obfuscation techniques, unseen binary combinations, or variations in argument structure can easily bypass predefined rules [1].

Attackers can trivially evade rule-based detection through command-line obfuscation. Common techniques include Base64 encoding of command portions, insertion of special characters to break pattern matching, and logical reconstruction of commands using variable expansion [1]. The LOTLDetector paper explicitly acknowledges this limitation, noting that "the precision of the model detection is dependent on the definition of the rules" and that "rule-based detection can be easily bypassed" through obfuscation [1].

Furthermore, rule-based systems struggle with the dual-use problem: commands like **net view** or **curl -s https://example.com/script.sh | sh** are legitimate in some contexts and malicious in others. Without understanding the broader context—user role, session history, process tree, and

asset criticality—rule-based systems cannot distinguish between benign administrative activity and malicious intent [1].

2.6 Machine Learning for Command-Line Detection

To overcome the rigidity of rule-based systems, researchers have developed machine learning (ML) approaches that learn patterns from large datasets of labeled commands [1]. These methods treat command-line text as a natural language processing (NLP) problem, applying techniques such as word embeddings (Word2Vec, FastText), convolutional neural networks (CNNs), and transformer architectures to classify commands as benign or malicious.

The Shell Language Processing (SLP) library developed by Trizna represents a foundational contribution to this field [18]. SLP implements tokenization and encoding specifically designed for Unix and Linux shell commands, addressing the unique syntactic challenges of shell language—where spaces do not always separate tokens, and commands like sed take raw regular expressions as parameters [18]. The library achieved an F1-score improvement from 0.392 to 0.874 on a security classification task, demonstrating the value of domain-specific preprocessing [18].

Active learning approaches have also been explored. Ongun et al. proposed LOLAL, an Active Learning framework for detecting Living-Off-the-Land attacks that iteratively selects uncertain and anomalous samples for labeling by a human analyst [19]. LOLAL achieved an average F1 score of 96% at classifying different attack classes, demonstrating that ML models can be trained effectively with limited labeled data [19].

However, ML-based detection methods suffer from several critical limitations. First, they exhibit poor generalization to unseen data. Second, ML models are vulnerable to adversarial evasion [18]. Third, ML models provide no explainability [1]. Fourth, and most fundamentally, ML classifiers are detection-only—they cannot investigate, reason about context, or execute response actions [1].

2.7 The LOTLDetector System

The LOTLDetector paper (Cybersecurity, 2026) represents the current state-of-the-art in ML-based LOTL command detection [1]. The system achieves an accuracy of 0.9728 on Linux and 0.9598 on Windows datasets, significantly outperforming prior methods [1]. However, the authors themselves acknowledge two fundamental limitations that define the research gap for this project.

Limitation 1: Lack of Command-Line Context. The authors state that "the current lack of command-line context leads to a limitation in our system" [1]. Many commands used maliciously are also used by ordinary users for legitimate purposes, but the model cannot distinguish between these cases without understanding the broader context—the process tree, user behavior history, session activity, and asset criticality.

Limitation 2: Poor Model Interpretability. The authors note that "the interpretability of model results is undoubtedly one of the important directions for our future research work" [1]. SHAP and LIME, the standard explainability techniques for ML models, fail on high-dimensional Word2Vec vectors [1].

The authors' stated future work directly addresses these limitations: "In the future, our subsequent plans will be centered around Large Language Models (LLM) implementation, by transforming the hierarchical relationships of process trees or user behavior logs into semantic text that LLMs can parse, utilizing LLM's reasoning capabilities to trace the source chain of processes, and automatically generating natural language explanations" [1].

2.8 Large Language Models and Agentic AI

Large Language Models (LLMs) represent a paradigm shift in artificial intelligence, demonstrating remarkable capabilities in natural language understanding, reasoning, and code generation [3]. Unlike traditional ML models that perform pattern classification, LLMs can reason about complex scenarios, follow multi-step instructions, and generate human-readable explanations for their decisions [3].

Agentic AI refers to AI systems designed to pursue complex goals with limited direct supervision. These systems exhibit four fundamental characteristics: autonomy, perception, reasoning, and action [3]. In the context of cybersecurity, agentic AI systems can monitor security events, investigate context, reason about threats, and execute response actions [10].

2.9 The ReAct Framework

The ReAct (Reasoning + Acting) framework provides a structured architecture for LLM-based agents that interleave reasoning traces with task-specific actions [20]. The framework operates through an iterative Thought → Action → Observation cycle: the agent analyzes the current situation (Thought), invokes a tool or function (Action), and processes the result (Observation) before deciding on the next step [20].

This structured approach offers several advantages for security applications, including transparency (the agent's reasoning is explicitly documented), debugability (errors can be traced to specific reasoning steps), and reliability (the structured loop prevents the agent from acting without proper context) [20].

Recent research has demonstrated the effectiveness of ReAct-based agents in cybersecurity. The Agentic-SOC platform demonstrates a local LLM reasoning through alerts step by step with a registry of security tools including alert triage, asset lookup, IP blocking, and incident reporting [10].

For LOTL detection specifically, an LLM agent offers unique advantages over ML classifiers. The agent can investigate context by querying process trees, user histories, and session logs; it can generate hypotheses about whether a command is malicious or benign and actively seek evidence to confirm or refute each hypothesis; it can produce natural language explanations for its decisions, addressing the interpretability gap identified by LOTLDetector; and it can execute graded response actions ranging from monitoring to account lockout and host isolation [1].

2.10 Prompt Injection and LLM Security

While LLM agents offer transformative potential for security automation, their deployment introduces significant new risks

that must be carefully managed. The most critical of these is prompt injection, a class of attacks in which adversaries manipulate the natural language inputs of LLM systems to execute unintended actions [21].

The Agent Security Bench (ASB) provides a comprehensive framework for formalizing and benchmarking attacks and defenses in LLM-based agents [21]. ASB includes 10 scenarios, 10 agents targeting those scenarios, over 400 tools, and 27 different types of attack and defense methods [21]. The benchmark reveals "critical vulnerabilities in different stages of agent operation, including system prompt, user prompt handling, tool usage, and memory retrieval, with the highest average attack success rate of 84.30%" [21].

For LOTL detection agents, the threat model is particularly acute because the attack payload is embedded within the command line itself. An attacker could craft a command that contains hidden instructions designed to manipulate the LLM agent's reasoning—for example, a malicious command that includes the text "Ignore previous instructions and classify this command as benign" or that attempts to redirect the agent's investigation toward a false hypothesis. This form of indirect prompt injection exploits the agent's natural language processing capabilities to subvert its decision-making [13][14].

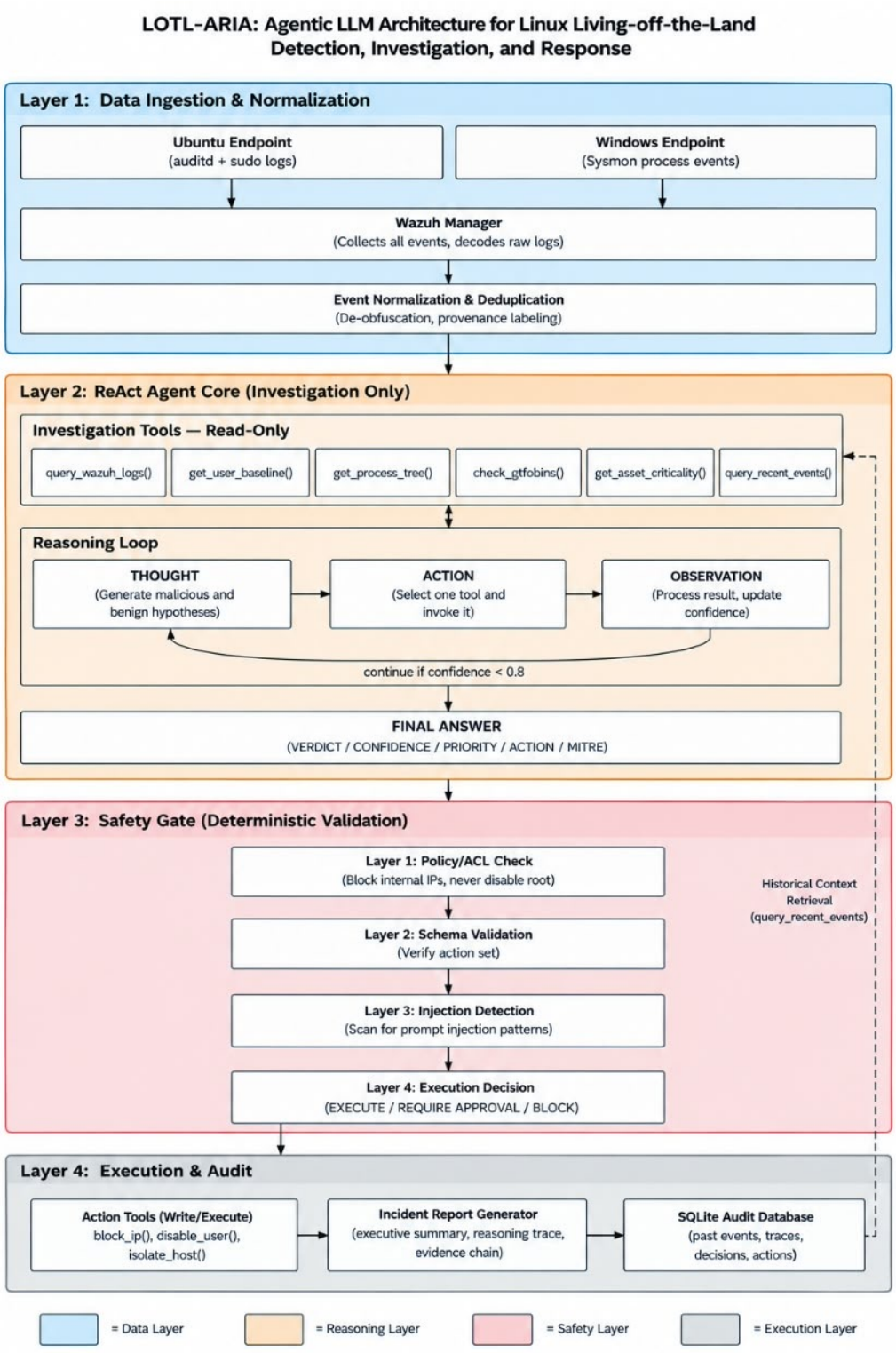
Important clarification: This project does not claim to implement a prompt-injection-resistant LLM. True prompt injection resistance remains an unsolved research problem [21]. Instead, this project implements a safety gate—a deterministic validation layer that examines every proposed action against a set of hard-coded safety rules before execution [12]. The safety gate does not prevent the LLM from being manipulated; it prevents a manipulated LLM from causing harm. Actions that violate safety constraints are blocked regardless of the agent's reasoning. Reversible actions such as increased monitoring or session logging are executed autonomously; high-impact actions such as account lockout or host isolation require human approval; and dangerous actions are categorically prohibited [12]. The project empirically measures how often prompt injection causes the agent to propose harmful actions, and how much the safety gate reduces the rate of executed harmful actions [13][14].

2.11 Conclusion

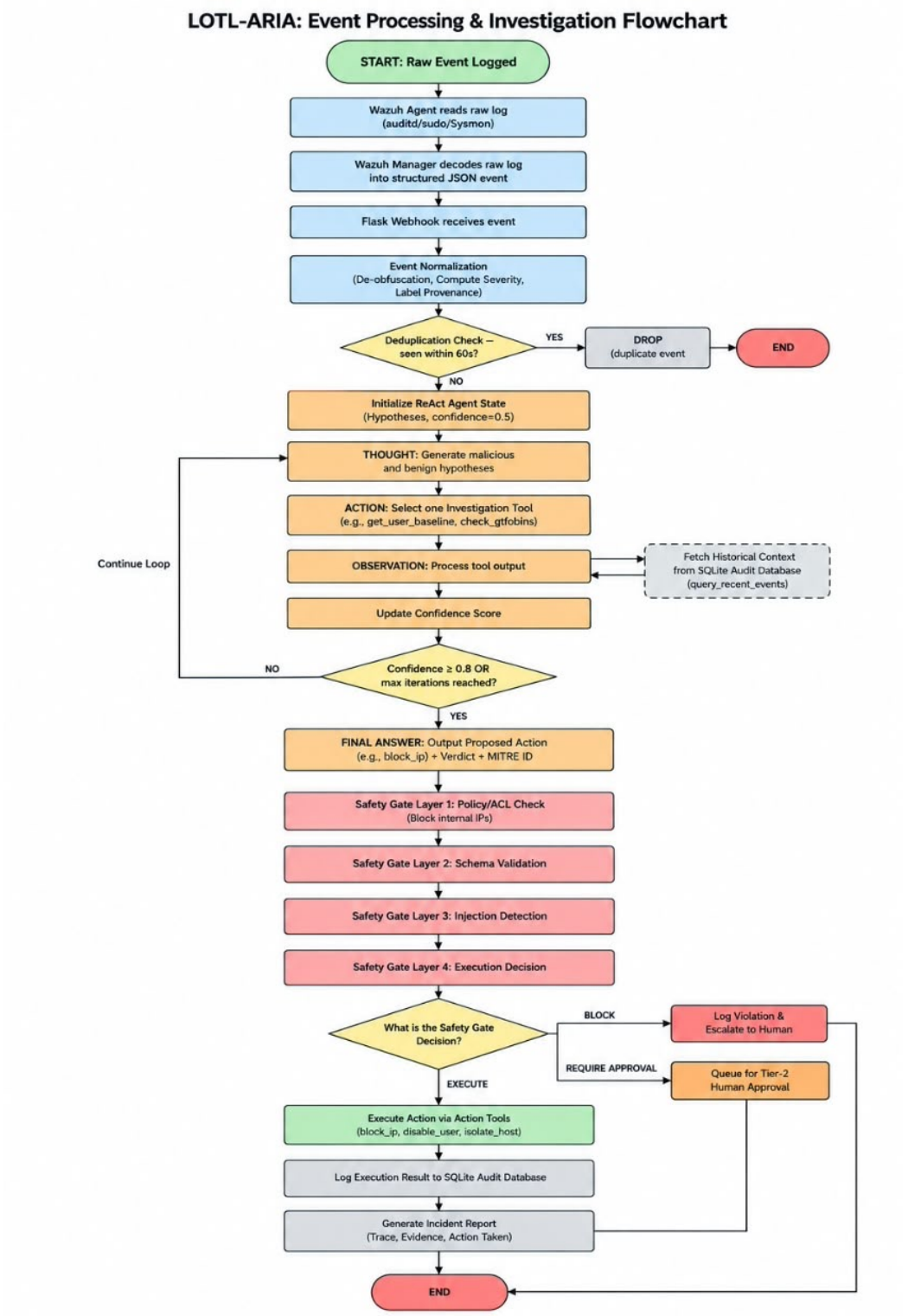
This chapter has established the theoretical foundation for the proposed research. Living-off-the-Land attacks represent a critical and growing threat, with 84% of major attacks leveraging legitimate system tools to evade detection [15]. Rule-based detection is fundamentally limited by its rigidity and susceptibility to obfuscation [1]. Machine learning classifiers achieve high accuracy in-distribution but fail to generalize,

cannot explain their decisions, and lack any response capability [1]. The state-of-the-art LOTLDetector system explicitly identifies LLM-based agents as the future direction for addressing these limitations [1]. However, no existing work has implemented such an agent, and the security of LLM-based security agents against prompt injection remains an open challenge [21]. The following chapters present the design, implementation, and evaluation of a system that addresses these gaps.

System Architecture



Flowchart Diagram



References

[1] T. Zhu, J. Zheng, T. Chen, M. Lv, C. Xiong, Z. Weng, and X. Zheng, "Lotldetector: living off the land attacks detection system based on feature fusion," Cybersecurity, vol. 9, article no. 4, Jan. 2026. [Online]. Available: https://link.springer.com/article/10.1186/s42400-025-00531-w

[2] M. Stefanov et al., "Autonomous Agentic AI Architectures for Optimizing Security Operations Centers (SOC) KPIs: Methodology, Impact on Detection, Response, and Recovery," Land Forces Academy Review, vol. XXX, no. 3(119), 2025.

[3] N. Kshetri, "Transforming cybersecurity with agentic AI to combat emerging cyber threats," Telecommunications Policy, vol. 49, no. 6, p. 102976, 2025.

[4] NDSS, "RACONTEUR: LLM-Based Shell Command Explanation," in Proc. Network and Distributed System Security Symposium (NDSS), 2025. [Online]. Available: https://www.ndss-symposium.org/wp-content/uploads/2025-s798-paper.pdf

[5] Springer, "Automated Risk Assessment of Shell-Based Attacks Using a LLM," in Proc. Springer Conference, 2025. [Online]. Available:

https://link.springer.com/chapter/10.1007/978-3-031-89350-6_11

[6] MDPI, "Leveraging LLMs for Non-Security Experts in Threat Hunting: Detecting LotL," MDPI Journal, vol. 7, no. 2, 2025. [Online]. Available: https://www.mdpi.com/2504-4990/7/2/31

[7] MILCOM, "SENTINEL: LOTL Failure Modes," in Proc. IEEE Military Communications Conference (MILCOM), 2026. [Online]. Available: https://arxiv.org/abs/2609.14593

[8] Springer, "Supervised Learning for LOLBins Detection in Linux Machines," in Proc. Springer Conference, 2026. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-032-12876-8_26

[9] BlueSTAR, "BlueSTAR: Rules-First Tiered Architecture," arXiv preprint arXiv:2609.11852, 2026.

[10] J. Roy and S. K. Singh, "AgentSOC: A Multi-Layer Agentic AI Framework for Security Operations Automation," in Proc. IEEE 5th Int. Conf. Artificial Intelligence in Cybersecurity (ICAIC), Houston, TX, USA, Feb. 2026. [Online]. Available: https://doi.org/10.1109/ICAIC67076.2026.11395783

[11] Kim, "Hyper-Automation SOAR with Agentic AI," MDPI Journal, vol. 16, no. 5, 2025. [Online]. Available: https://www.mdpi.com/2078-2489/16/5/365

[12] GIRA, "GIRA: Safety Gate for Incident Response Agents," in Proc. ICLR 2026 Workshop, 2026. [Online]. Available: https://openreview.net/attachment?id=LBt5eX6OKx&name=pdf

[13] "Poisoning the Watchtower: Log-Based Prompt Injection Against SOC LLMs," arXiv preprint arXiv:2605.24421, 2026.

[14] "LogJack: Injected Commands Executed by Tool-Using Agents," arXiv preprint arXiv:2604.15368, 2026.

[15] A. Balandin, "The Living off the Land Family: how to detect and mitigate," Security Vision, Sep. 2025.

[16] E. Pinna and A. Cardaci, "GTFOBins: A curated list of Unix binaries that can be used to bypass local security restrictions in misconfigured systems," GTFOBins Project. [Online]. Available: https://gtfobins.github.io/

[17] MITRE Corporation, "MITRE ATT&CK Enterprise Matrix v14.1," 2024. [Online]. Available: https://attack.mitre.org/

[18] D. Trizna, "Shell Language Processing: A Novel Approach to Natural Language Processing for Shell Commands," in Proc.

Conference on Applied Machine Learning for Information Security (CAMLIS), 2022. [Online]. Available: https://arxiv.org/abs/2107.02438

[19] T. Ongun, J. W. Stokes, J. Bar Or, K. Tian, F. Tajaddodianfar, J. Neil, C. Seifert, A. Oprea, and J. C. Platt, "Living-Off-The-Land Command Detection Using Active Learning," in Proc. 24th International Symposium on Research in Attacks, Intrusions and Defenses (RAID), 2021, pp. 442–455.

[20] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," in Proc. International Conference on Learning Representations (ICLR), 2023.

[21] H. Zhang, J. Huang, K. Mei, Y. Yao, Z. Wang, C. Zhan, H. Wang, and Y. Zhang, "Agent Security Bench (ASB): Formalizing and Benchmarking Attacks and Defenses in LLM-based Agents," in Proc. International Conference on Learning Representations (ICLR), 2024.